

模板展开 (templating)

【题目背景】

西西艾弗岛信息中心主持了岛上的信息化工作，为各个部门提供信息化服务。不同部门所需要的信息化服务内容各有不同，但是其中存在着一个共同的需求：通过模板生成文档。为了满足这个需求，信息中心开发了一个通用的文档模板系统。模板系统有若干输入参数，这些参数都是字符串，模板系统根据模板，将这些字符串拼接为最终的输出。为了让模板系统能够支持更复杂的需求，信息中心决定开发一种模板语言，这种模板语言重点支持变量和其代换。你的任务是能够读取这种模板语言，并按照模板语言的规则生成相应输出。

【题目描述】

模板语言涉及到的数据是字符串。我们约定，字符串是由零个或多个小写字母和数字 (`[a-z0-9]`) 组成的序列。模板语言中的变量的名称是非空字符串，且变量的值也是字符串。在模板开始执行前，所有变量的值都是空字符串。

模板语言中的表达式涉及对变量的值和字符串进行拼接操作。表达式由空格分隔的若干个操作数组成，操作数可以是变量或非空字符串；其中如果操作数的第一个字符是美元符号 (`$`)，则表示该操作数是一个变量，操作数的剩余部分为该变量的名称；否则，表示该操作数是一个字符串。表达式的值是指先将表达式中的所有变量替换为其值，然后将所有操作数拼接在一起得到的字符串。

例如，假设有变量 `a` 的值为 `hello`，变量 `b` 的值为 `world`，则：

- 表达式 `a` 的值为 `a`；
- 表达式 `$a` 的值为 `hello`；
- 表达式 `a b` 的值为 `ab`；
- 表达式 `$a b` 的值为 `hello b`；
- 表达式 `$a $b` 的值为 `helloworld`；
- 表达式 `$a $b a` 的值为 `helloworlda`。

模板语言由一系列语句组成，每个语句占一行，每个语句按顺序执行。语句分为两类：赋值语句和输出语句。其中，赋值语句用于将变量和表达式关联起来，输出语句用于输出变量的值。赋值语句分为两种：直接赋值和间接赋值。

直接赋值语句的形式为空格分隔的数字 `1`、变量名称和表达式，形如 `1 <var> <expr>`，表示求表达式的值并将该值作为变量的值。例如，语句 `1 c $a $b` 执行后，变量 `c` 的值为 `helloworld`。

间接赋值语句的形式为空格分隔的数字 `2`、变量名称和表达式，形如 `2 <var> <expr>`，表示暂时不求表达式的值，而是将该表达式与变量相对应。待到后续求其它表达式的值需要用到该变量时，再求该变量对应的表达式的值并将该值作为变量的值用于求所求的表达式。例如考虑以下模板：

```

1 1 a hello
2 1 b world
3 2 c $a $b
4 1 d good $c
5 1 a hi
6 1 e good $c

```

在执行第三个语句 `2 c $a $b` 时，变量 `c` 与表达式 `$a $b` 对应起来，但并不求表达式的值。在执行第四个语句 `1 d good $c` 时，需要求表达式 `good $c` 的值，并将该值作为变量 `d` 的值。此时需要用到变量 `c`，因此需要先求表达式 `$a $b` 的值，即 `helloworld`，然后再和 `good` 拼接得到 `goodhelloworld`。因此，执行完第四个语句后，变量 `d` 的值为 `goodhelloworld`。在执行第五个语句 `1 a hi` 时，变量 `a` 的值被更新为 `hi`。在执行第六个语句 `1 e good $c` 时，需要求表达式 `good $c` 的值，需要用到变量 `c`，此时仍要重新求表达式 `$a $b` 的值，即 `hiworld`，然后再和 `good` 拼接得到 `goodhiworld`。因此，执行完第六个语句后，变量 `e` 的值为 `goodhiworld`。需要注意的是，虽然变量 `c` 没有被重新赋值，但由于 `c` 是间接赋值的，所以在每次用到 `c` 时都要重新求表达式 `$a $b` 的值，即 `c` 的值是动态的。

输出语句的形式为空格分隔的数字 3 和变量名称，形如 `3 <var>`，表示输出一行，内容为变量的值的长度除以 1000000007 的余数。例如，语句 `3 d` 表示输出变量 `d` 的值的长度。用输出语句修改上述例子的模板：

```

1 1 a hello
2 1 b world
3 2 c $a $b
4 3 c
5 1 a hi
6 3 c

```

将会输出：10 和 7，对应的变量的值为：

```

1 helloworld
2 hiworld

```

需要注意的是，在使用间接赋值语句时，在变量的值之间建立了依赖关系，在上述模板中，变量 `c` 的值依赖于变量 `a` 和 `b` 的值。可以想见，如果变量的值间的依赖关系形成了环，那么模板将无法执行。我们约定，给定的表达式中不存在这样的环。

形式化地，模板语言用 BNF 表示如下：

```

1 CHAR ::= [a-z0-9]
2 SPACE ::= ' '

```

```
3 DOLLAR ::= '$'  
4 NONEMPTY_STRING ::= CHAR | NONEMPTY_STRING CHAR  
5 STRING ::= '' | NONEMPTY_STRING  
6 VAR ::= NONEMPTY_STRING  
7 OPERAND ::= DOLLAR VAR | NONEMPTY_STRING  
8 EXPR ::= OPERAND | EXPR SPACE OPERAND  
9 ASSIGN_OP ::= '1' | '2'  
10 ASSIGN ::= ASSIGN_OP SPACE VAR SPACE EXPR  
11 OUTPUT ::= '3' SPACE VAR  
12 STATEMENT ::= ASSIGN | OUTPUT
```

【输入格式】

从标准输入读入数据。

输入的第一行为一个整数 n ，表示模板语言的语句数量。接下来的 n 行，每行为一个语句，语句的格式如上所述。

【输出格式】

输出到标准输出。

依次执行输入的语句：如果语句为输出语句，则相应输出一行变量的值的长度除以 10000000007 的余数。

【样例 1 输入】

```
1 6  
2 1 a hello  
3 1 b world  
4 2 c $a $b  
5 3 c  
6 1 a hi  
7 3 c
```

【样例 1 输出】

```
1 10  
2 7
```


【样例 1 解释】

本样例即为题目描述中的例子。

【样例 2 输入】

```
1 5
2 1 var value
3 3 var
4 3 val
5 1 var $var $val $var
6 3 var
```

【样例 2 输出】

```
1 5
2 0
3 10
```

【样例 2 解释】

执行第一条语句后，变量 **var** 的值为 **value**，因此第二条语句输出 **5**；执行第三条语句时，变量 **val** 并未被赋值，因此其值为初始的空字符串，输出 **0**；执行第四条语句后，变量 **var** 的值为 **valuevalue**，因此第五条语句输出 **10**。

【子任务】

测试点	n	语句类型	表达式的性质
1, 2	≤ 200	无间接赋值语句	每个表达式仅有 1 个字符串操作数
3, 4			每个表达式仅有 1 个操作数
5, 6		包含所有类型语句	每个表达式包含不超过 100 个操作数
7, 8			
9, 10	≤ 2000		

全部的数据满足：变量的总数不超过 1000 个，且每个变量的名称、字符串的长度不超过 50 个字符。